ARC UI · UIARC.DEV · PROOF OF CONCEPT

# Agent-Buildable Showcase

Testing whether Arc UI can be built end-to-end from its free, unauthenticated AI-facing surface

Generated 2026-11-11

A proof of concept that tests the claim "The React UI library your AI agents can build with" by having an agent discover Arc UI through its AI-facing surface and assemble a working "Create project" screen from genuine upstream Arc source.

---

## Overview

Arc (https://uiarc.dev) positions itself as a React component and block library designed for AI-assisted frontend development. This POC tests whether that positioning holds in practice. It characterizes the surfaces Arc exposes to machines, then plays the role of an AI coding agent: it reads the index, learns each component's API from its markdown docs, installs the real source from the public registry, and compiles the result.

**Thesis:** an agent with web access can build a real UI from Arc using only Arc's free, unauthenticated AI surface, end to end, from discovery to compiling code.

**What this POC proves:**

- Arc's `llms.txt` index gives an agent a complete component catalog and explicit rules written for agents.
- Per-component markdown docs expose typed prop tables that an agent can compose against.
- The public shadcn registry ( `/r/<name>.json` ) serves the actual MIT source, so "read about it" and "install it" are the same pipeline.
- The composed screen type-checks and builds ( `npm run build` , as reported in `WRITEUP.md` and `README.md` ), with no invented props.

**What it does not prove:** that the MCP server is usable by an unattended agent (it is OAuth-gated), or that Pro components are available without a paid account.

Source material: `WRITEUP.md` (fetched excerpts, MCP probe, agent-build transcript), fetched on 2026-11-09/10 with read-only GET requests plus one harmless MCP `initialize` / `tools/list` POST.

---

## Arc's AI-facing surface

Arc advertises several machine-readable surfaces from `llms.txt` . The table below records what each returns, its HTTP status and auth in this test, and what an agent gets from it.

| Surface | URL | Status / auth | What it returns | What an agent gets |
|---|---|---|---|---|
| **llms.txt** (index) | `https://uiarc.dev/llms.txt` | **200**, no auth. About 90 KB. | Site summary, install notes, a **"Rules for agents"** section, skills, guides, comparisons, templates, docs, and the full component and block catalog. Each entry is a link with a one-line "job" description. | Discovery: what exists, how it is grouped, and the contract the agent must follow (prefer Arc components, read markdown before use, use only documented props, use semantic tokens, keep reduced-motion branches). |
| **llms-full.txt** | `https://uiarc.dev/llms-full.txt` | Advertised by llms.txt. **Not fetched** in this test. | Per the index, every component's markdown inlined. | A single-file context dump (not exercised). |
| **llms-small.txt** | `https://uiarc.dev/llms-small.txt` | Advertised by llms.txt. **Not fetched** in this test. | Per the index, one line per item. | A compact catalog (not exercised). |
| **Per-component markdown** | `https://uiarc.dev/components/<name>/markdown` | **200**, no auth. | Structured docs: When to use / When not to use, Installation (CLI and manual), Usage snippet, **API reference** (typed prop table), Keyboard, Accessibility, Motion, Responsive, Performance, **Notes for AI**, Guidance for AI tools. | The exact API contract: prop names, types, defaults, and copy-ready usage. This is what the agent composes against. |
| **Per-block markdown** | `https://uiarc.dev/components/blocks/<name>/markdown` | **200**, no auth. | The same template, for multi-part interface blocks. | Block-level composition guidance. |
| **Registry item** | `https://uiarc.dev/r/<name>.json` | **200** for free items, no auth. MIT. | A standard shadcn registry item: `name`, `type`, `dependencies`, `registryDependencies`, and `files[]` with `path` and `target`, including the real source. | The genuine implementation, installable into `src/components/arc/`. |
| **Registry index / catalog** | `https://uiarc.dev/r/registry.json`, `https://uiarc.dev/r/catalog.json` | Advertised by llms.txt. **Not fetched** in this test. | Per the index, registry and catalog metadata. | Machine-readable inventory (not exercised). |
| **Agent skills** | `https://uiarc.dev/r/skills/arc/*.md` | Advertised by llms.txt. **Not fetched** in this test. | Per the index, agent skill files. | Additional agent guidance (not exercised). |
| **MCP server** | `POST https://uiarc.dev/api/mcp` | **401**, OAuth-gated. Bearer challenge with `resource_metadata`, scope `openid profile email`. Auth server: `https://clerk.uiarc.dev`. | JSON-RPC error `-32001`: "Sign in to use the Arc MCP server." `tools/list` returns the same 401. | Per llms.txt, tools for list, search, get, install, and skill. **Unavailable anonymously**, so the tool list could not be enumerated. |

### Notes on the surfaces

- **Rules for agents.** llms.txt contains instructions written to the agent, not the human. These rules are the contract the POC followed: prefer existing Arc components, read markdown first, use only documented props, use semantic tokens such as `--background`, `--surface`, `--foreground`, `--text-secondary`, `--border`, `--accent`, `--success`, `--warning`, and `--danger` instead of raw colors, support dark mode via `data-theme="dark"`, and keep a reduced-motion branch for every animation.
- **Catalog size.** Arc has **243 components and 141 blocks**. **121 components and 22 blocks are free and open source (MIT)**. **122 components and 119 blocks are Arc Pro**, which install from the token-gated `@uiarc-pro` registry.
- **Install path.** `components.json` in this repo registers the `@uiarc` namespace as `https://uiarc.dev/r/{name}.json`. `npx shadcn@latest add @uiarc/<name>` performs the same fetch the POC performed manually.

## The agent-build flow

The POC followed this pipeline, acting as the agent with Arc's AI surface as the only input:

```
discover (llms.txt)  →  learn API (per-component markdown)  →  install real source (registry)  →  compile
```

| Step | Action | Arc surface | Outcome |
|---|---|---|---|
| 1. Discover | `GET /llms.txt` | llms.txt | Learned that Arc exists, that components are grouped by job, that `components.json` can register `@uiarc`, and read the "Rules for agents" contract. |
| 2. Decide and select | Map the screen's needs to the index's one-line descriptions | llms.txt index | Chose Input, Textarea, Select, Switch, and Button for a "Create project" settings screen. All five are free (MIT). |
| 3. Learn the API | `GET /components/{input,textarea,select,switch,button}/markdown` | Per-component markdown | Extracted typed prop tables. Examples: Select takes `options: {value, label, disabled?}[]` and `onValueChange`; Switch uses `checked` and `onCheckedChange`; Input and Textarea take `label`, `description`, and `error`. |
| 4. Install real source | `GET /r/<name>.json`, write each `files[].content` to `files[].target` (`@components` maps to `src/components`) | Registry | Real `.tsx` and `.module.css` files landed in `src/components/arc/<name>/`, along with the shared foundation tokens and motion tokens each item declares. |
| 5. Assemble | Compose the components into `src/App.tsx` using only documented props, plus tokens from `foundation.css` for app chrome | Agent code | A working "Create project" screen. |
| 6. Verify | `tsc -b && vite build` | Toolchain | Reported green. The documented APIs matched the real source, and no prop was invented. |

**Key observation from the transcript:** because each registry item ships its real source, the agent does not have to infer behavior from docs alone. The type-check in step 6 is the check that the documented props match the shipped code.

## What was assembled

The POC builds a single **"Create project"** settings screen (`src/App.tsx`). It has:

- Two text inputs: project name (with live validation: at least 3 characters, reworded in place) and an auto-suggested URL slug.
- A description textarea with a live "N characters left" counter (max 280).
- Two selects: visibility (Private, Team, Public) and hosting region (United States, Europe, Asia Pacific).
- Three switches: usage analytics, preview environments, and a dark-mode toggle that sets `data-theme` on `<html>`.
- Two buttons: a primary "Create project" button (uses `loading` while saving, and is disabled until the name is valid) and a ghost "Cancel" button.
- On submit, the screen simulates a save for 1.2 seconds, then renders the project payload as JSON.

### Arc components composed

| Arc component | Instances in `App.tsx` | Props used | Source file |
|---|---|---|---|
| `Input` | 2 | `label`, `value`, `onChange`, `description`, `error`, plus native `placeholder` and `autoFocus` | `src/components/arc/input/input.tsx` |
| `Textarea` | 1 | `label`, `value`, `onChange`, `rows`, `maxLength`, `description` | `src/components/arc/textarea/textarea.tsx` |
| `Select` | 2 | `label`, `value`, `onValueChange`, `options`, `description` | `src/components/arc/select/select.tsx` |
| `Switch` | 3 | `label`, `checked`, `onCheckedChange` | `src/components/arc/switch/switch.tsx` |
| `Button` | 2 | `variant` (`primary`, `ghost`), `loading`, `disabled`, `onClick` | `src/components/arc/button/button.tsx` |

### Provenance

| Artifact | Provenance | Notes |
|---|---|---|
| `src/components/arc/button/*`, `input/*`, `textarea/*`, `select/*`, `switch/*` | **Real upstream Arc source**, fetched from `https://uiarc.dev/r/<name>.json` | Per `WRITEUP.md`, unmodified. |
| `src/components/arc/foundation.css` | **Real upstream Arc source** (design tokens) | Imported once in `src/main.tsx`. |
| `src/components/arc/motion-tokens.ts` | **Real upstream Arc source** (registry dependency) | Per `WRITEUP.md`. |
| `src/components/arc/lib/motion-tokens.ts` | Not listed in the `WRITEUP.md` provenance table. It is a one-line re-export ( `export { motionTokens } from "../motion-tokens"` ) that the component files import. | Its comment says it is a "shared alias used by the gallery." Treat its origin as unrecorded. |
| `src/App.tsx`, `src/index.css`, `src/main.tsx` | **Hand-written** by the agent | `App.tsx` composes the components with documented props only. `index.css` is app chrome built on semantic tokens. |
| `components.json`, `vite.config.ts`, `tsconfig*.json`, `package.json`, `index.html` | **Hand-written scaffold** | Mirrors Arc's documented `@uiarc` setup and standard Vite and TypeScript conventions. |

---

## Assessment: does the pitch hold up?

**Verdict: yes for the free llms.txt + markdown + registry loop, with an asterisk on MCP and a paywall behind most of the catalog.**

**What holds up**

- **Discoverability.** llms.txt gives an agent the full catalog, one-line job descriptions, and an explicit contract. The agent did not have to guess what existed.
- **Docs built for composition.** Typed prop tables, Notes for AI, and copy-ready usage snippets give the agent a contract to compose against. Every documented prop used in this POC matched the real source.
- **The loop closes.** The registry serves real MIT source publicly. The POC went from discovery to compiling code without authentication.
- **Guardrails are machine-usable.** Semantic tokens, `data-theme` dark mode, and reduced-motion branches are stated as rules and also embodied in the shipped source.

**Caveats**

- **MCP is OAuth-only.** The MCP server returns 401 to anonymous and headless clients. A real MCP client would run an OAuth sign-in flow, which an unattended agent cannot complete. The genuinely frictionless surface for an autonomous agent is llms.txt, markdown, and the registry. MCP is a convenience layer over the same content and the gate for Pro source.
- **Pro is most of the library.** 122 of 243 components and 119 of 141 blocks are Pro and token-gated. The free tier is enough for real screens like this one, but "build anything from the catalog" depends on a paid account.
- **Dependency weight.** Even simple components pull in `motion`. Select and Switch also pull in Radix. The bundle is heavier than hand-rolled controls, which is a trade-off for motion and accessibility quality.

**Bottom line:** For an agent with web access, Arc is buildable end to end from its free, unauthenticated AI surface. That is a stronger position than a typical component library whose agent story is "here are some docs." MCP is a useful addition, not the thing that makes the loop work.

---

## Project structure

```
agent-buildable/
├── README.md               # Entry point: what the POC is, how to run it, links to WRITEUP
├── WRITEUP.md              # Primary evidence: fetched excerpts, MCP probe, transcript, assessment
├── package.json            # Scripts (dev, build, preview) and dependencies
├── package-lock.json       # Locked dependency tree
├── components.json         # shadcn config with the @uiarc registry entry
├── index.html              # Vite HTML entry point
├── vite.config.ts          # React plugin and the @ -> src alias
├── tsconfig.json           # TypeScript project references
├── tsconfig.app.json       # App compiler options (strict, @ path mapping, bundler resolution)
├── tsconfig.node.json      # Config for Vite tooling
├── .gitignore              # Ignores node_modules, dist, env files, and tsc build info
├── dist/                   # Production build output (generated, gitignored)
└── src/
    ├── main.tsx            # Mounts <App/>; imports foundation.css, then index.css
    ├── App.tsx             # The "Create project" screen (the agent-assembled code)
    ├── index.css           # App chrome only, built on semantic tokens
    └── components/
        └── arc/            # Arc source, pulled unmodified from uiarc.dev/r/*.json
            ├── foundation.css       # Arc design tokens: color, space, radius, type, motion, focus, accents
            ├── motion-tokens.ts     # Motion presets (durations, easings, springs, blur)
            ├── lib/
            │   └── motion-tokens.ts # One-line re-export of ../motion-tokens (provenance not recorded)
            ├── button/              # button.tsx + button.module.css
            ├── input/               # input.tsx + input.module.css
            ├── textarea/            # textarea.tsx + textarea.module.css
            ├── select/              # select.tsx + select.module.css (Radix Select)
            └── switch/              # switch.tsx + switch.module.css (Radix Switch)
```

---

## Getting started

Requirements: Node.js and npm. The repo does not pin a Node version in `package.json`.

```
# Install dependencies
npm install

# Start the Vite dev server
npm run dev

# Type-check and produce a production build (tsc -b && vite build)
npm run build

# Serve the production build locally
npm run preview
```

To re-fetch the Arc source for a component, use the same registry path the POC used:

```
npx shadcn@latest add @uiarc/<name>
```

or fetch `https://uiarc.dev/r/<name>.json` directly and write each `files[].content` to its `files[].target`.

---

## Notes and limitations

- **Snapshot in time.** The Arc source and docs were fetched on 2026-11-09/10. The registry may change, so re-fetching can produce different files.
- **Build result not re-run for this document.** The "build passes" result comes from `WRITEUP.md` and `README.md`. This documentation pass did not re-execute the build.
- **Unverified provenance details.** `src/components/arc/lib/motion-tokens.ts` has no provenance entry in `WRITEUP.md`. The `.module.css` files were not diffed against the registry in this pass. `WRITEUP.md` states they are unmodified.
- **Surfaces not exercised.** `llms-full.txt`, `llms-small.txt`, the registry index and catalog, and the agent skills were advertised but not fetched. The MCP tools were not enumerated (401).
- **Pro components not used.** The screen uses only free components. Pro source requires a token.
- **Simulated save.** "Create project" does not call a backend. It waits 1.2 seconds and renders the payload as JSON.
- **Cancel reloads the page.** The ghost "Cancel" button calls `window.location.reload()`. A real app would navigate or reset state.

- **Slug behavior.** The slug auto-fills from the name until the user edits the slug field, after which it stops syncing.
- **Dark-mode control markup.** The dark-mode `Switch` sits inside a `<label>` that also contains the text "Dark", and the `Switch` has its own `aria-label` of "Dark mode". Review the accessible name if this control is productionized.
- `components.json` **contents.** It includes standard shadcn init fields, such as `style`, `tailwind`, and `aliases` for `@/lib/utils` and `@/hooks`. Those alias targets do not exist in this repo, which only contains `src/components/arc/` and `src/main.tsx` and `src/App.tsx` under `src/`.
- **Dependencies.** The bundle includes `motion`, `@radix-ui/react-select`, `@radix-ui/react-switch`, and `lucide-react`. This is intentional for motion and accessibility, but it adds weight compared with hand-rolled controls.
- **Scope of the claim.** The POC shows that an agent can build from the free surface. It does not show that an agent can build anything in the catalog, or that MCP works for an unattended agent.